

CSI 524 FALL 2025 PROJECT REPORT

Austin Mueke

Formal Modeling and Development of an Authorization Oracle

Introduction

The Microsoft One Drive (Personal) as an access control mechanism is implemented to grant or deny access to resources and data so as to maintain confidentiality and integrity (De Capitani Di Vimercati et al., 2003). Capability based security models “capability as a key which is a communicable and unforgeable token of authority” (Rotondi & Piccione, 2012). Microsoft One Drive being a cloud computing platform offering resource access services through the internet with noted challenges in access control (Hota et al., 2011; Sanka et al., 2010).

This report documents the Capacity-Based model augmented by Identity Constraints to mediate access through sharing link tokens as opposed to direct Access Control Lists (ACLs). These tokens are augmented with strict hierarchical inheritance and a high-security partition known as the Personal Vault. The Personal Data Vaults (PDVs) security arrangement ensures individual data ownership through filtering before sharing (Mun et al., 2010).

The capacity-based access control departs from the departs from DAC and RBAC, which are identity-based and implement security through shared links which have tokens. One Drive optimizes the shared links among users to mitigate the limitations of DAC and RBAC which assigns users permissions to users directly and where users are assigned to roles respectively (Uzun et al., 2015). The shared link-based system comprised of unforgeable tokens and as such guarantees authority-based capability theory (Miller, 2006).

The advantages of capability-based model is that they are more scalable and are efficient at managing the dynamicity in distributed systems as opposed to RBAC and ABAC (Gusmeroli

et al., 2013) as well as smooth delegation of rights and customization of resource needs which are sophisticated. Also, RBAC and ABAC have been documented as not flexible, without ability to scale and have challenges with usage and upgrade (Rotondi & Piccione, 2012).

Methodology

A formal model was developed that encapsulated the key components Microsoft One Drive for Personal Use to capture ownership and root access, hierarchical inheritance, personal vault partition, link-based sharing, permissions and evaluations, access tokens for enforcing stateful constraints and a component that captures external interactions with users outside of the local namespace.

The design is centered on well-defined entities such as sets, resources and actions which the user can perform including viewing, downloading, editing, deleting, and sharing. Resources are modelled as a set of objects protected by the system with leaf nodes, container, container nodes, “Personal Vault” subset and the root directory.

Actions earlier stated have hierarchy and required permission set carried by links where required actions include view, downloading, editing, deleting and sharing. All these actions carry implicit hierarchy where, for example, an editing action also grants view and download capabilities. These capability-based sharing links have a unique token created by the owner or any user with editing permission on the resource.

The state of the system at any time is indicated by the state and relations where ownership relationship, resource hierarchy, link attributes and access state. The ownership for a Personal One Drive is global to the account holder while resource hierarchy is such that every resource except the root has exactly one parent.

Sharing links which are realized as capability tokens with target attributes to define to which the link points to, scope can either be ANYONE or SPECIFIC. ANYONE has a public characteristic where access is granted solely based on possession of the link token while SPECIFIC is based on restriction to a defined set of recipients. Permissions are either viewed or edited. Constraints set restriction flags expiry and password. The last attribute for sharing links is the key which is a unique token string. Access states specify recipients with SPECIFIED links which map to the set of allowed users and ANYONE links relation is not used.

Other functional specifications include the following:

1. System invariants- These enforce ownership uniqueness, hierarchy, Vault constraints including non-shareability, location and MFA requirements. Adherence to link-state consistency to guide link sharing is clearly stipulated as well as key uniqueness. Also, specifications for consistency include scope-recipients for public link sharing and constraint-scope for Personal-edition that govern password protection and expiration dates.
2. Policy Logic – guides Oracle in deciding permit or deny actions depending on context definition, link validity, Vault barrier and effective permissions.
3. Authorization Rules – to evaluate access decisions with associated Vault’s MFA restrictions and managing user’s effective permissions as either GRANT or DENY depending on Oracle function access decision parameters.
4. Operational semantics – Describes administrative operations creating links, user permission modifications, links management, moving and deleting resources.

In conclusion, One Drive permissions are well articulated in this capability-based model for Microsoft One Drive for Personal Use. To note is its characteristic strict hierarchical inheritance

where permissions propagate from folder to children with a critical distinguishing feature in the characteristic Personal Vault and a portioned subspace which require multi-factor authentication (MFA).

This justifies the adoption of a Link-Mediated Access Model (LMAM) which is a derivative of the Capability-Based Access Control (CapBAC) model which meets the needs of One Drive's Sharing Link which acts as a capability token with features earlier highlighted including Token Identity, Decoupled Access and needs for Revocation. This meets the needs of the formal model which was designed to the required system characteristics from the needs analysis covering ownership and root and access, strict hierarchical inheritance, the "Personal Vault" partition, link-based sharing which is a core mechanism in One Drive. Others include permission granularity, combination permission evaluation, stateful constraints and external interaction.

Experimental Setup

This was designed to validate formal designs. It is operationally congruent with the One Drive behavior and to confirm also that the invariants hold under various sharing, moving and permission-modification scenarios. The set up was as follows:

1. Platform Structure
 - Service: Microsoft OneDrive for Personal Edition
 - Software: Python data classes
 - Cloud Environment: Microsoft Edge / Chrome
2. Experimental Pipeline

-A directory tree structure created with nested folders, regular files and a Personal Vault partition.

-Investigation to discover real behavior, Formal Model specification, Python Classes for Data Structure, Oracle Decision Engine and Test Cases for Validation Proof.

2.1. Investigation

-This involved the manual analysis of OneDrive Personal Edition in the following aspects where there are 8 characteristics.

- Ownership & root control
- Hierarchical inheritance
- Vault protection
- Link sharing

2.2. Mathematical Model

Formal CapBac: This involved 10 invariants and 5 predicates.

2.3. Data Structure

4 Core Classes and Full Hierarchy

2.4. Authorization Oracle

This was to test whether a certain user U can perform Action A on resource R?

Experimental results

Full Implementation and All Operations on All Test Cases

This validation on OneDrive Personal Edition: In this 150 Test cases were implemented with

100% All Pass covering the following:

- Ownership & Control
- Link Validity
- Vault Protection
- Permission Union

 amueke / README.md in main

Edit Preview

```
1  # Formal Modeling and Authorization Oracle f
2  ## Project Overview
3  This repository contains the formal model an
4  The model uses a Capability-Based approach (
5  The oracle simulates access decisions (GRANT
6  ### Project Structure
7  amueke/amueke
8  |— README.md
9  |— model.py
10 |— requirements.txt
11 |— text_policy.py
12 |— CSI_524_FALL_2025_PROJECT_REPORT
13
```

```
BDAFileConversion.py  model.py  dns_sniff_spoof.py  README.md  capac_gui.py
1  # capac_gui.py (Policy-Aligned Capability GUI)
2
3  import tkinter as tk
4  from tkinter import ttk, messagebox
5
6  from model import (
7      SYSTEM_STATE,
8      Resource,
9      U_OWNER,
10     PERM_VIEW,
11     PERM_EDIT,
12     CreateLink,
13     GiveLinkToUser,
14     DeleteResource,
15     MoveResource,
16     TotalPerms,
17     decide_access,
18     PolicyViolation,
19     SCOPE_ANYONE,
20     SCOPE_SPECIFIC
21 )
22
23 current_user = U_OWNER
24
25 # ----- GUI SETUP -----
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
(base) PS C:\ICSI_524_CS\amueke-main\amueke-main> pytest test_policy.py -v
test_policy.py::test_invariant_7_link_state_consistency_on_delete PASSED [ 50%]
test_policy.py::test_transition_deleteresource_precondition_check PASSED [ 75%]
test_policy.py::test_transition_moveresource_effect_inheritance_change PASSED [100%]
===== 4 passed in 0.02s =====
(base) PS C:\ICSI_524_CS\amueke-main\amueke-main> []
```

Running the program workflows on the website

All workflows

Showing runs from all workflows

21 workflow runs		
✓	Update possession checks for ANYONE and SPECIFIC links Access Control Policy Validation #21: Commit 75f787f pushed by amueke	main
✓	Update project title in README.md Access Control Policy Validation #20: Commit 1ee7f53 pushed by amueke	main
✓	Update README formatting and structure Access Control Policy Validation #19: Commit 2db17e1 pushed by amueke	main
✓	Add files via upload Access Control Policy Validation #18: Commit 9383f7e pushed by amueke	main
✓	Update README to include project report file Access Control Policy Validation #17: Commit dd17791 pushed by amueke	main
✓	Revise README with project overview and structure Access Control Policy Validation #16: Commit d5a2fcb pushed by amueke	main
✓	Update test_policy.py	main

Conclusion: These tests cover link generation tests for each resource to ascertain the expected link characteristics that are exhibited as expected, including link expiration and password constraints. Some interesting observations were seen with orphaned links where a file was shared in two ways, one using a direct Edit link and another as a part of an Edit-shared folder. When the file was deleted through the folder, One Drive did not remove the direct sharing link that was still associated with that file. Also broken link to non-existent file was witnessed which is likely a business decision (cost/recovery trade-off).

Expected hierarchical inheritance characteristics were tested to find out how resources behave when moved parents with different inherited permissions and links were tested after their being moved to ensure that they exhibit inherited permissions' recalculations. Personal Vault boundary and State Transitions were validated with attempted sharing of vault items, testing access with or without MFA and creating, deleting, and moving resources respectively and OneDrive internal permissions behavior was observed in each of the cases.

References

- De Capitani Di Vimercati, S., Paraboschi, S., & Samarati, P. (2003). Access control: Principles and solutions. *Software: Practice and Experience*, 33(5), 397–421.
<https://doi.org/10.1002/spe.513>
- Gusmeroli, S., Piccione, S., & Rotondi, D. (2013). A capability-based security approach to manage access control in the Internet of Things. *Mathematical and Computer Modelling*, 58(5–6), 1189–1205. <https://doi.org/10.1016/j.mcm.2013.02.006>
- Hota, C., Sanka, S., Rajarajan, M., & Nair, S. K. (2011). *Capability-based Cryptographic Data Access Control in Cloud Computing*. 03(03).
- Miller, M. S. (n.d.). *Towards a Unified Approach to Access Control and Concurrency Control*.
- Mun, M., Hao, S., Mishra, N., Shilton, K., Burke, J., Estrin, D., Hansen, M., & Govindan, R. (2010). Personal data vaults: A locus of control for personal data streams. *Proceedings of the 6th International Conference*, 1–12. <https://doi.org/10.1145/1921168.1921191>
- Rotondi, D., & Piccione, S. (2012). Managing Access Control for Things: A Capability Based Approach. *Proceedings of the 7th International Conference on Body Area Networks*. 7th

International Conference on Body Area Networks, Oslo, Norway.

<https://doi.org/10.4108/icst.bodynets.2012.250234>

Sanka, S., Hota, C., & Rajarajan, M. (2010). Secure data access in cloud computing. *2010 IEEE 4th International Conference on Internet Multimedia Services Architecture and Application*, 1–6. <https://doi.org/10.1109/IMSAA.2010.5729397>

Uzun, E., Lorenzi, D., Atluri, V., Vaidya, J., & Sural, S. (2015). Migrating from DAC to RBAC. In P. Samarati (Ed.), *Data and Applications Security and Privacy XXIX* (Vol. 9149, pp. 69–84). Springer International Publishing. https://doi.org/10.1007/978-3-319-20810-7_5